Autómatas Studio

Editor, simulador y conversor de DFA a Expresión Regular mediante eliminación de estados

Documentación del proyecto

Estudiante	Nombre del estudiante
Curso	Lenguajes Formales y Autómatas
Fecha	Octubre de 2026
Repositorio	https://github.com/usuario/automata-studio
Tecnología	Java 17 · Swing · JGraphX 4.2.2 · Maven

1. Resumen del proyecto

Autómatas Studio es una aplicación de escritorio en Java que permite **dibujar un autómata finito determinista (DFA)** sobre un lienzo interactivo, **simular** cadenas con animación, **convertirlo a una expresión regular** mediante el algoritmo de eliminación de estados (con animación paso a paso) y **verificar la equivalencia** entre el DFA y la regex usando el motor `java.util.regex`. Con ello se aplica la equivalencia entre lenguajes regulares, autómatas finitos y expresiones regulares.

Funcionalidades principales

- **Lienzo interactivo:** crear estados con clic, conectar transiciones (incluidos bucles y aristas opuestas), marcar estado inicial y de aceptación, mover, renombrar, editar y eliminar.
- **Simulación:** recorrido animado símbolo a símbolo, resaltando el estado actual y la transición usada; veredicto ACEPTADA / RECHAZADA.
- **Conversión a regex:** normalización (q_start, q_accept), eliminación iterativa con la fórmula $R_{ik}' = R_{ik} \cup (R_{ij} \cdot (R_{jj})^* \cdot R_{jk})$, animación del desvanecimiento de nodos y etiquetas actualizadas, y resultado final.
- **Equivalencia:** prueba de una cadena en ambos modelos y verificación exhaustiva de todas las cadenas hasta una longitud dada.

2. Arquitectura del software

El proyecto sigue el patrón **MVC** con una separación estricta en cuatro paquetes. La regla de dependencia es: el **modelo** y los **algoritmos** no conocen ninguna clase gráfica (no importan Swing ni JGraphX), los **controladores** coordinan y la **vista** solo dibuja y reporta eventos.

Paquete	Responsabilidad	Clases principales
model	Representación matemática A = (Q, Σ, δ, q0, F). Sin lógica gráfica.	State, Transition, Automaton, Edge, Examples
algorithm	Lógica pura: álgebra de regex, grafo GNFA, eliminación de estados, simulación, equivalencia. Totalmente testeable sin GUI.	RegexOps, GnfaGraph, StateEliminator, EliminationStep, DfaSimulator, EquivalenceChecker
controller	Traducen gestos del usuario en cambios del modelo y orquestan las animaciones (javax.swing.Timer).	EditorController, SimulationController, EliminationController, EquivalenceController
view	Componentes Swing + JGraphX. Dibujan lo que se les entrega y notifican eventos mediante interfaces.	MainFrame, GraphView, SimulationPanel, ConversionPanel, EquivalencePanel

Desacoplamiento mediante interfaces

Los controladores no dependen de paneles concretos: `SimulationController.SimulationView` y `EliminationController.EliminationView` son interfaces implementadas por los paneles, y `GraphView.Listener` permite que el lienzo avise de clics, arrastres y teclas sin conocer al `EditorController` que los atiende. Los pasos del algoritmo (`EliminationStep`) son records con instantáneas del grafo: la animación solo los reproduce.

Diagrama de clases UML

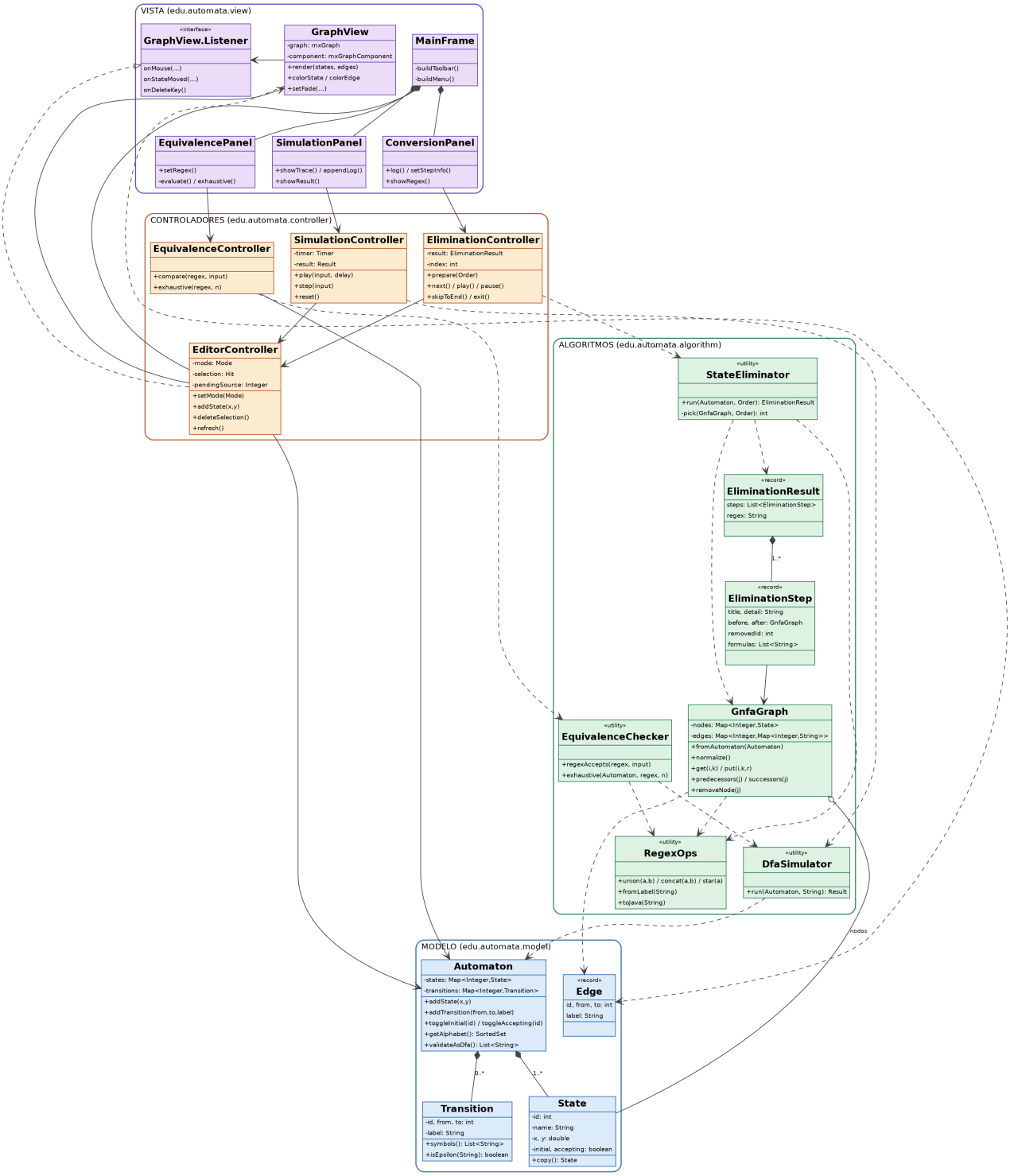


Figura 1. Diagrama de clases. Rombo lleno = composición, rombo vacío = agregación, flecha continua = asociación, flecha punteada = dependencia, triángulo punteado = realización de interfaz.

3. Estructuras de datos y algoritmo de reducción

3.1 Representación del autómata (modelo)

`Automaton` guarda los estados y las transiciones en dos `LinkedHashMap<Integer, …>` indexados por identificador. Se eligió LinkedHashMap porque ofrece acceso, inserción y borrado en O(1) y conserva el orden de creación (útil para la estrategia «orden de creación» y para mostrar siempre los estados en el mismo orden). Las transiciones referencian estados por id (no por objeto), lo que simplifica borrar un estado: basta eliminar las transiciones cuyo origen o destino sea ese id. El alfabeto Σ no se almacena: se **infiere** de las etiquetas.

3.2 Grafo generalizado GnfaGraph

Para el algoritmo se usa un grafo cuyas aristas llevan **expresiones regulares** en lugar de símbolos. Se implementa como **lista de adyacencia con mapas anidados**:

```
Map<Integer, State>                  nodes   // q_start = -1, q_accept = -2
Map<Integer, Map<Integer, String>>   edges   // edges.get(i).get(k) = R_ik   (sin entrada ⇒ ∅)
```

Esta estructura da acceso O(1) a R_{ik} y permite obtener los predecesores y sucesores de un estado sin recorrer toda la matriz. Una matriz n×n sería más simple pero desperdiciaría memoria en grafos dispersos (lo habitual en DFAs), y una lista de aristas haría costosa la consulta de R_{ik}. Las aristas paralelas del DFA se fusionan con unión (∪) al construir el grafo.

3.3 Álgebra de expresiones regulares (RegexOps)

Las expresiones son cadenas de texto con dos convenciones: `null` representa ∅ (lenguaje vacío) y `"ε"` representa la cadena vacía. Cada operación aplica las **identidades del álgebra** para no generar expresiones innecesariamente largas:

Operación	Identidades aplicadas	Ejemplo
union(a, b)	∅ ∪ R = R ; R ∪ R = R ; ε ∪ R = R? (o R* si ya es estrella)	union(ε, a) = a?
concat(a, b)	∅·R = ∅ ; ε·R = R ; paréntesis solo si hay unión de nivel superior	concat(a|b, c) = (a|b)c
star(a)	∅* = ε ; ε* = ε ; (R*)* = R* ; (R?)* = R*	star(ab) = (ab)*

3.4 Algoritmo de eliminación de estados

Paso 1 — Normalización. Se añade un estado inicial único q_start con una transición ε hacia q0 y un estado final único q_accept con transiciones ε desde cada estado de F. Así q_start no tiene aristas entrantes y q_accept no tiene salientes.

Paso 2 — Reducción iterativa. Para eliminar un estado intermedio j, cada par (i, k) con i ∈ pred(j) y k ∈ succ(j) se recalcula:

$$R_{ik}' = R_{ik} \cup (R_{ij} \cdot (R_{jj})^* \cdot R_{jk})$$

El término **$(R_{jj})^*$** es la aplicación del **Lema de Arden**: la ecuación $X = R_{jj} \cdot X \cup S$ tiene como solución mínima $X = (R_{jj})^* \cdot S$. Geométricamente, el bucle del estado eliminado se «absorbe» en las aristas que lo atravesaban.

Paso 3 — Animación. Por cada eliminación, `EliminationController` reproduce tres fases con un `javax.swing.Timer`: (a) resaltar el nodo a eliminar y sus aristas, (b) desvanecerlo reduciendo la opacidad, (c) dibujar el grafo resultante marcando en verde las aristas recalculadas.

Paso 4 — Resultado. Cuando solo quedan q_start y q_accept, la etiqueta de la única arista es la expresión regular buscada.

```
ELIMINAR_ESTADOS(DFA):
  G ← GNFA(DFA)                      // R_ik = unión de etiquetas; ∅ si no hay arista
  G.normalizar()                     // q_start -ε-> q0 ;  q ∈ F -ε-> q_accept
  mientras G tenga estados intermedios:
      j    ← elegir(G)               // el de menor |pred(j)| · |succ(j)|
      loop ← star(R_jj)              // Lema de Arden
      para cada i ∈ pred(j), k ∈ succ(j):
          R_ik ← union(R_ik, concat(concat(R_ij, loop), R_jk))
      G.eliminar(j)
  devolver R(q_start, q_accept)       // null = ∅
```

Orden de eliminación. El resultado es siempre correcto, pero su tamaño depende del orden. La estrategia «automática» elige el estado con menor producto |pred|·|succ| (heurística clásica que minimiza nuevas aristas); la alternativa recorre los estados en orden de creación. **Complejidad:** $O(n^3)$ operaciones sobre expresiones; el tamaño de la regex puede crecer exponencialmente en el peor caso, característica conocida del método.

3.5 Ejemplo trabajado: cantidad par de 0s (Σ = {0,1})

DFA: q0 (inicial y de aceptación), q1; δ(q0,0)=q1, δ(q0,1)=q0, δ(q1,0)=q0, δ(q1,1)=q1.

Paso	Acción	Resultado
1	Normalizar	q_start –ε→ q0 ; q0 –ε→ q_accept
2a	Eliminar q1 (pred = {q0}, succ = {q0}). R(q0,q0)' = 1 ∪ (0 · (1)* · 0)	bucle en q0 con etiqueta 1|01*0
2b	Eliminar q0 (pred = {q_start}, succ = {q_accept}). R' = ∅ ∪ (ε · (1|01*0)* · ε)	q_start –(1|01*0)*→ q_accept
4	Resultado	**R = (1|01*0)***

3.6 Simulación y verificación de equivalencia

`DfaSimulator.run` recorre la cadena y devuelve la lista de pasos (from, símbolo, transición, to); la animación solo reproduce esa lista, de modo que la lógica no depende de la interfaz. Antes de simular se valida que el autómata sea un DFA (estado inicial presente, sin ε y sin transiciones repetidas con el mismo símbolo). `EquivalenceChecker` compara el veredicto del DFA con `Pattern.compile(regex).matcher(cadena).matches()` para una cadena, o para **todas** las cadenas sobre Σ hasta una longitud máxima (por niveles, con tope de 200 000 cadenas), devolviendo un contraejemplo si lo hay.

4. Manual de usuario

4.1 Requisitos y ejecución

Se necesita **JDK 17 o superior** y **Apache Maven 3.8+**. Desde la carpeta del proyecto:

```
mvn clean package          # compila, ejecuta las pruebas y genera target/automata-studio-1.0.0.jar
java -jar target/automata-studio-1.0.0.jar

# alternativa sin generar el jar:
mvn compile exec:java
```

4.2 Pantalla principal

La ventana tiene una **barra de herramientas** (modos y acciones), el **lienzo** al centro y, a la derecha, tres pestañas: **1. Simulación**, **2. Regex** y **3. Equivalencia**. La barra inferior muestra ayuda contextual. Al abrir la aplicación se carga un ejemplo (cantidad par de 0s); también puede elegirse otro en Archivo → Cargar ejemplo.

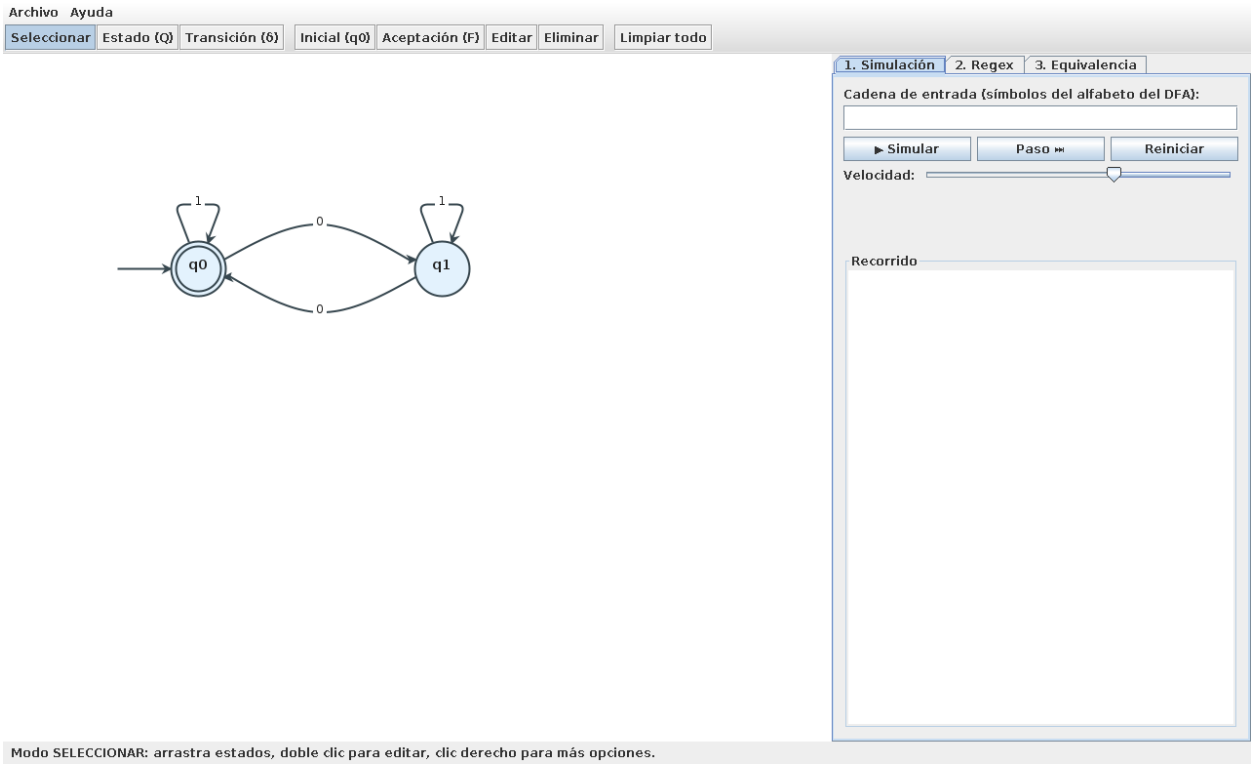


Figura 2. Ventana principal con el ejemplo «cantidad par de 0s» cargado. La flecha entrante marca el estado inicial; el doble círculo, el de aceptación.

4.3 Construir un autómata

- **Estado (Q):** activa el modo Estado (Q) y haz clic en un espacio vacío. El primer estado creado es el inicial por defecto.
- **Transición (δ):** activa Transición (δ), haz clic en el estado de origen y luego en el de destino (mismo estado = bucle). Escribe el símbolo en el cuadro que aparece. Esc cancela.
- **Propiedades:** selecciona un estado y usa Inicial (q0) o Aceptación (F) (o el menú contextual con clic derecho).

- **Editar:** en modo Seleccionar, arrastra los estados para moverlos; doble clic en un estado lo renombra y en una arista edita su etiqueta; Supr o Eliminar borra lo seleccionado.
- **Etiquetas:** un símbolo (a), varios (a,b), epsilon (eps o ε) o una expresión parcial (ab*).

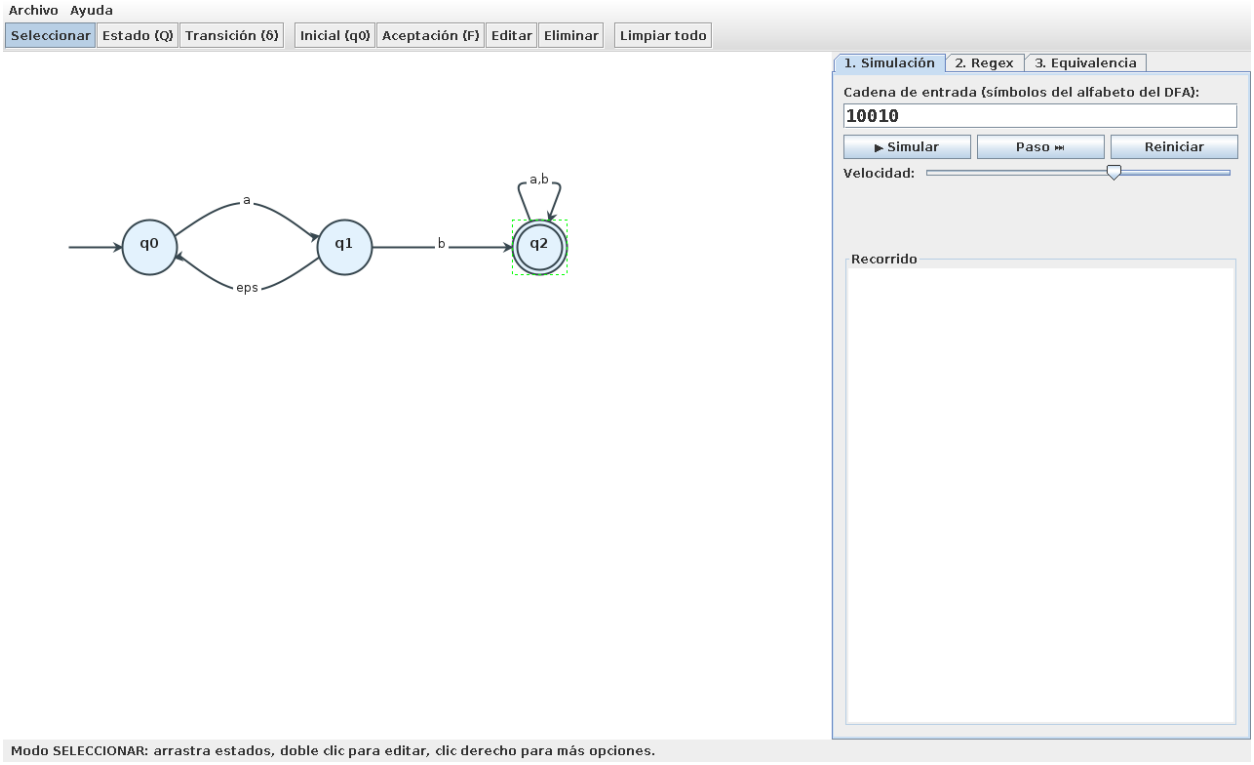


Figura 3. Autómata dibujado desde cero con el ratón: estados, transiciones, un bucle (a,b), una arista ε y un estado de aceptación.

4.4 Simulación de cadenas (pestaña 1)

Escribe una cadena y pulsa ▶ **Simular** (animación automática, velocidad regulable) o **Paso** ⏭ (un símbolo por clic). El estado actual se pinta de amarillo, la transición usada en rojo y la parte consumida de la cadena en verde. Al terminar se muestra **ACEPTADA** (verde) o **RECHAZADA** (rojo) y el estado final se colorea igual. Si el autómata no es un DFA válido, se indica el motivo (por ejemplo «falta el estado inicial» o «no determinista»).

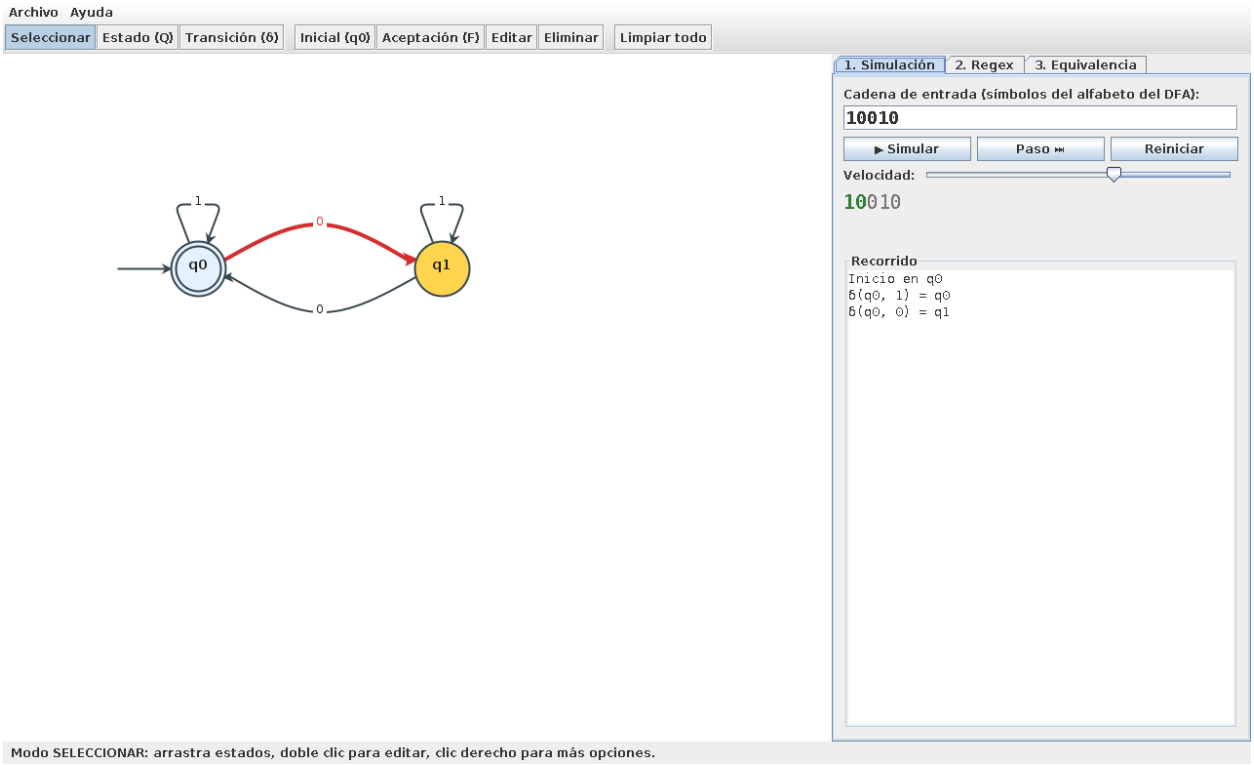


Figura 4. Simulación en curso con la cadena 10010: dos símbolos consumidos, estado actual q1.

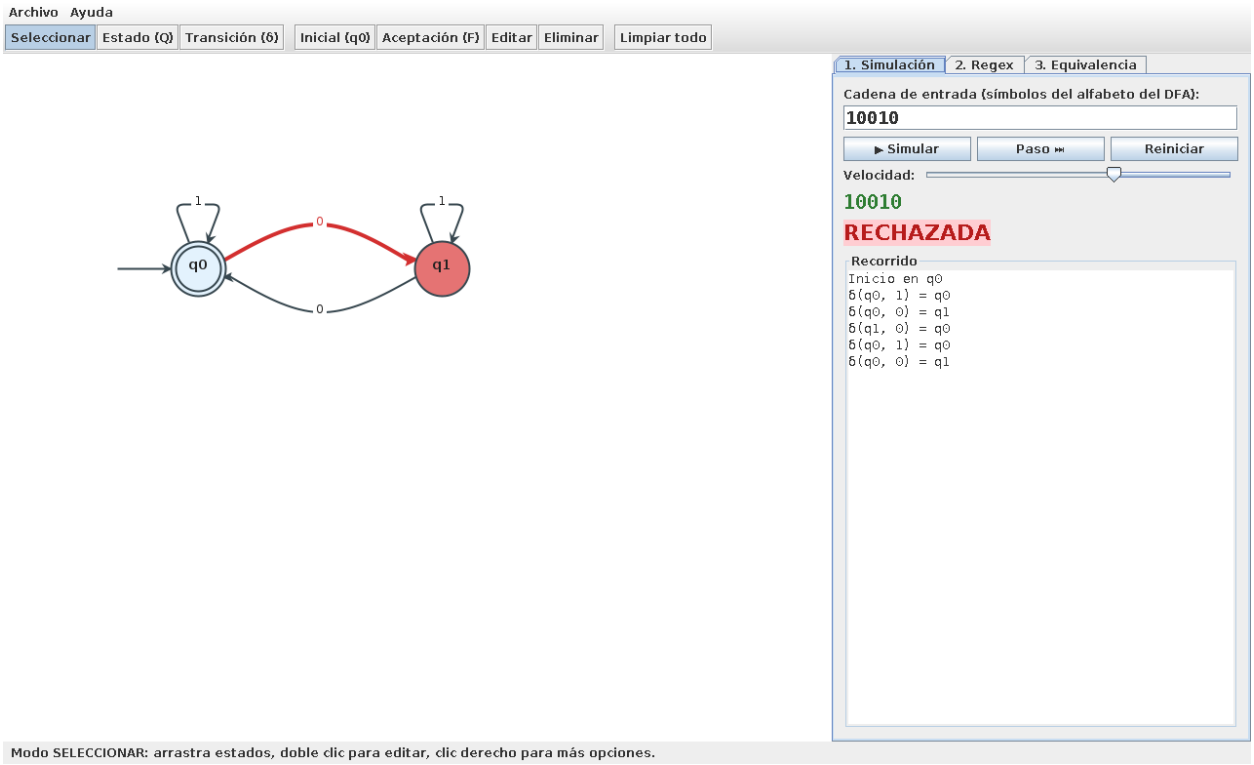


Figura 5. Fin de la simulación: la cadena 10010 tiene tres ceros, por lo que es RECHAZADA.

4.5 Conversión a expresión regular (pestaña 2)

Elige el orden de eliminación y pulsa **Iniciar**; el lienzo pasa a mostrar el grafo del algoritmo y se bloquea la edición. Avanza con **Siguiente paso**, reproduce todo con ▶ **Animar** (**Pausa** la detiene), salta con **Final** o regresa con ↩ **Volver al editor**. El cuadro «Pasos del algoritmo» registra cada fórmula R_{ik}' aplicada.

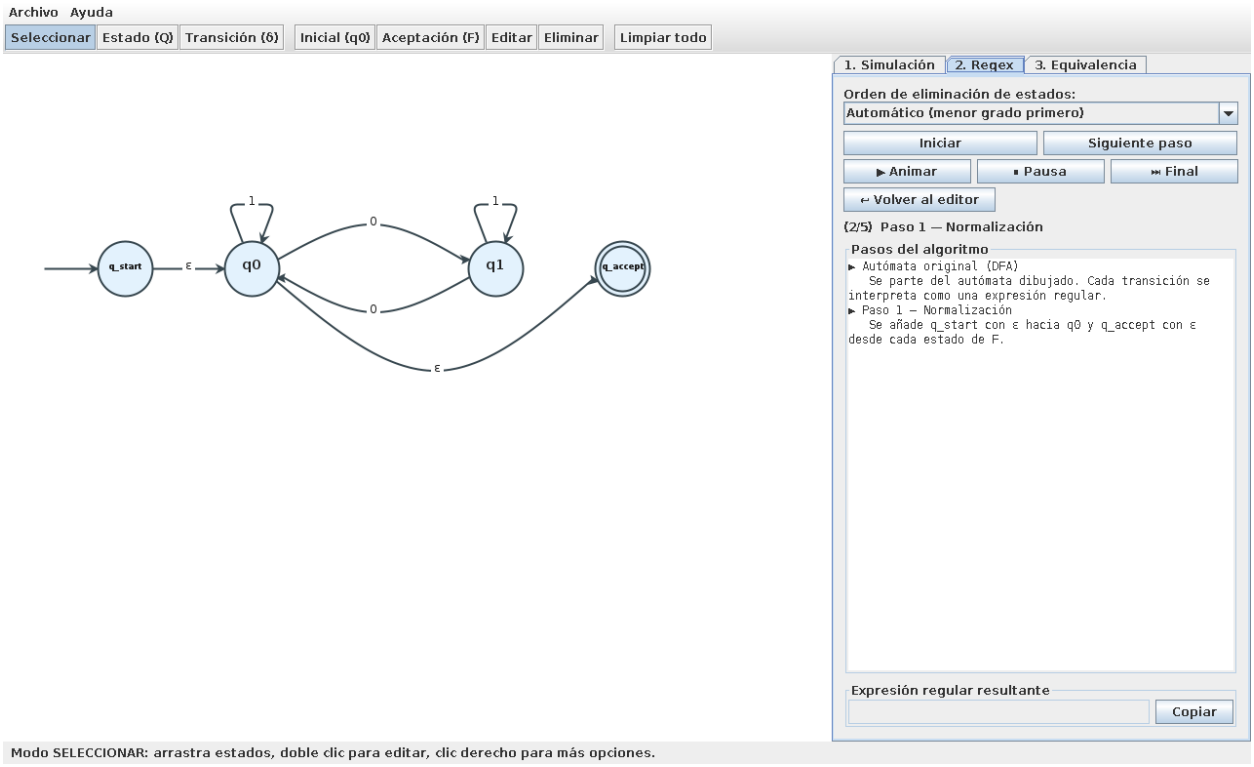


Figura 6. Paso 1 — Normalización: se añaden q_start (con ε hacia q0) y q_accept (con ε desde cada estado de F).

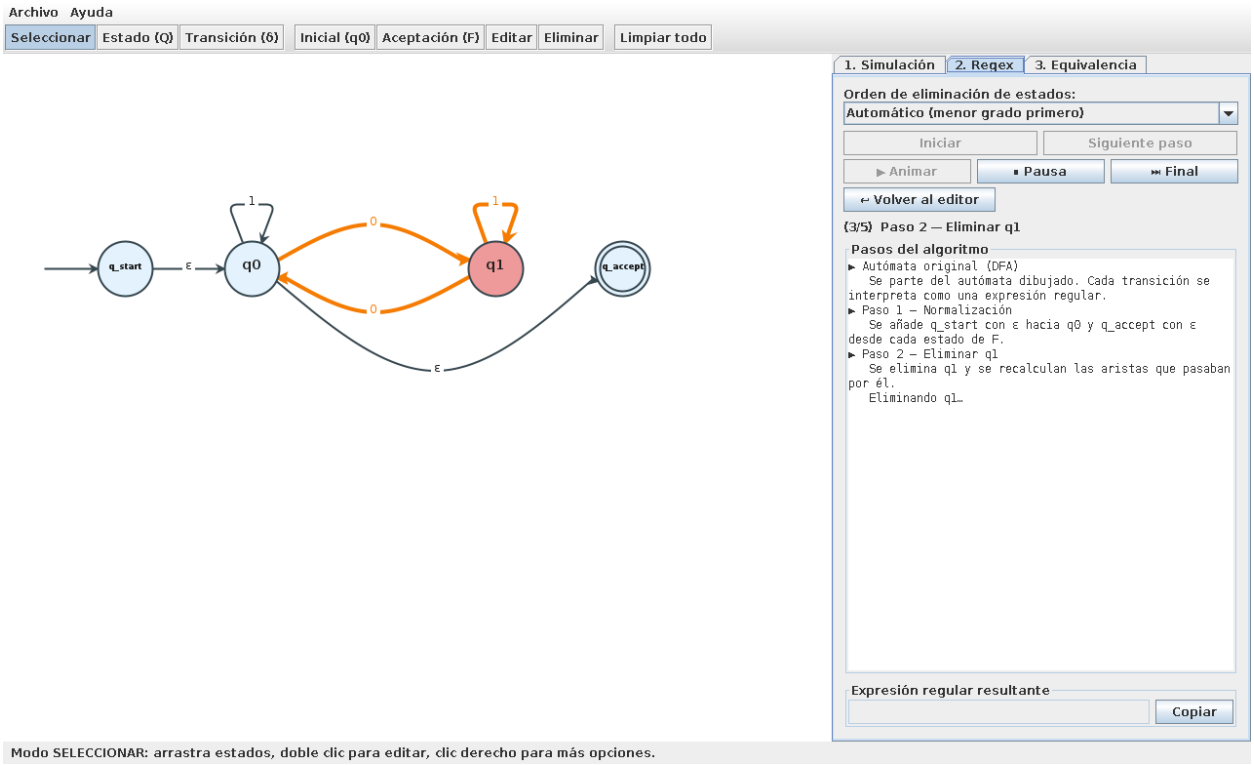


Figura 7. Paso 2 — Se resalta q1 (rojo) y las aristas que pasan por él (naranja) antes de desvanecerlo.

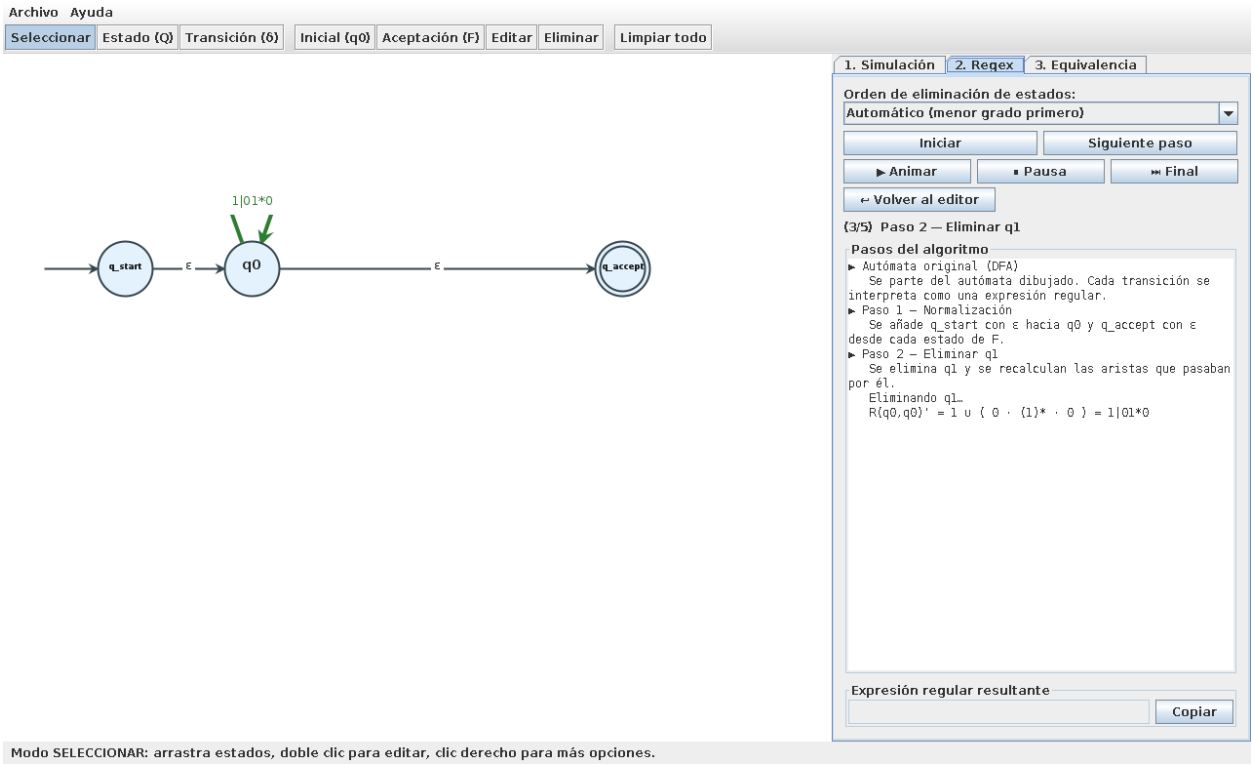


Figura 8. Tras eliminar q1: el bucle de q0 queda etiquetado 1|01*0 (verde = arista recalculada).

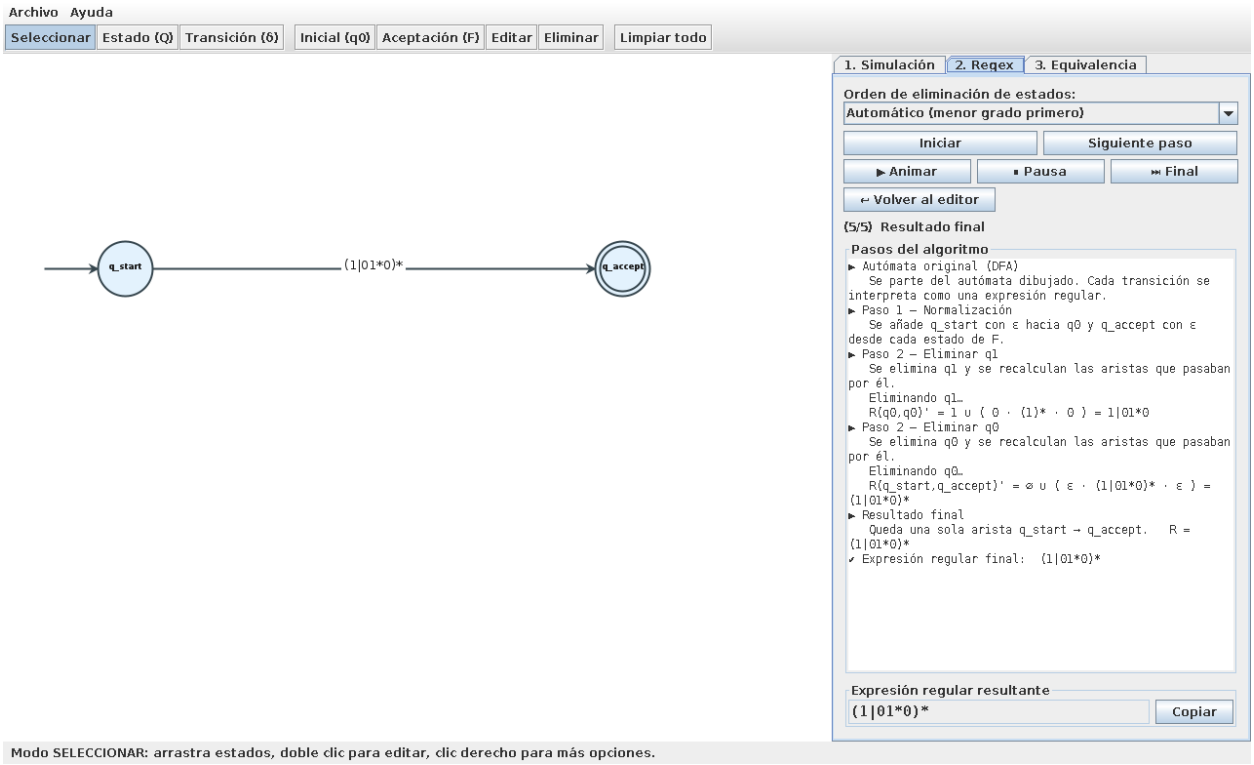


Figura 9. Resultado final: una sola arista q_start → q_accept con la expresión (1|01*0)*, copiable con el botón «Copiar».

4.6 Verificación de equivalencia (pestaña 3)

Escribe una cadena de prueba y pulsa **Evaluar en DFA y en la regex**: se muestra el veredicto de cada modelo y si coinciden. **Verificación exhaustiva** prueba todas las cadenas hasta la longitud elegida y reporta equivalencia o un contraejemplo. Si se edita el autómata, la regex deja de ser válida y debe regenerarse.

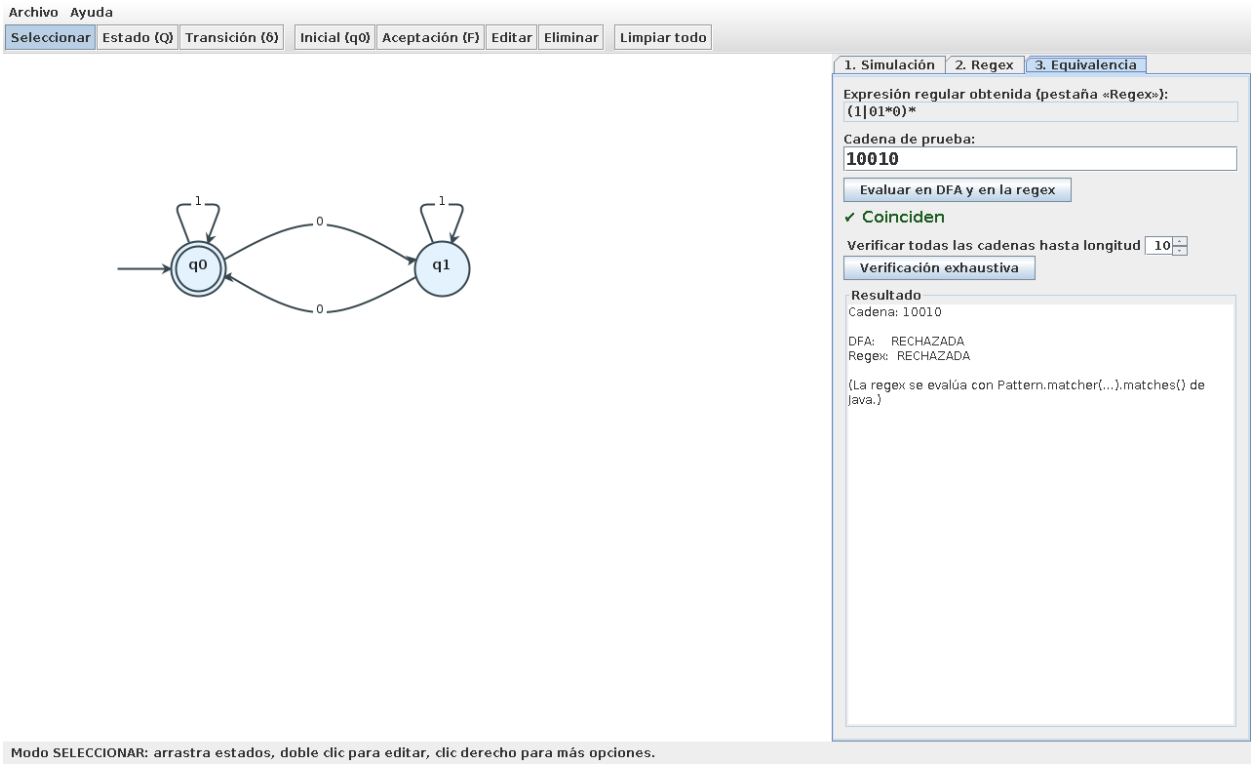


Figura 10. Evaluación de la cadena 10010: ambos modelos la rechazan (✔ Coinciden).

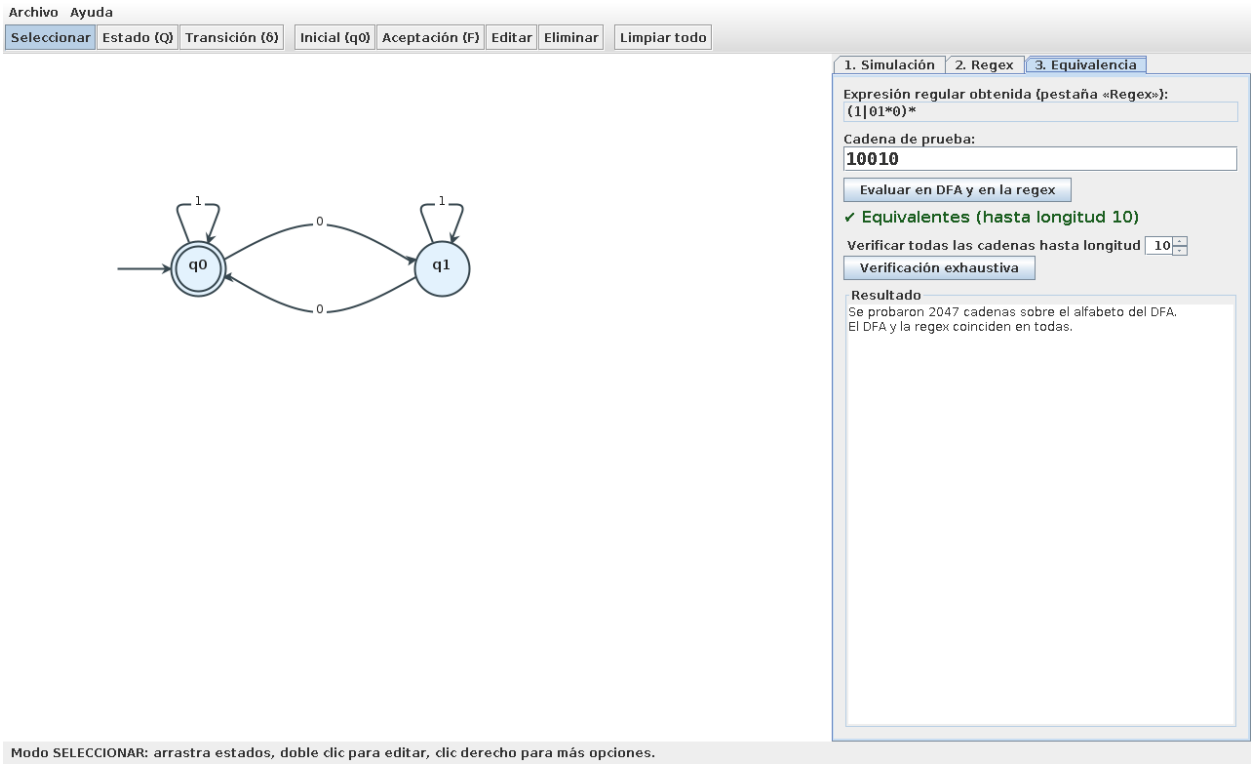


Figura 11. Verificación exhaustiva: 2047 cadenas (hasta longitud 10) sin ningún contraejemplo.

4.7 Problemas frecuentes

Mensaje / síntoma	Causa y solución
«Falta marcar el estado inicial»	Selecciona un estado y pulsa «Inicial (q0)».
«No determinista: q0 tiene más de una transición con 'a'»	Un DFA permite solo una transición por símbolo y estado; corrige la etiqueta.
«La etiqueta … no es un símbolo único»	Las expresiones parciales (ab*) sirven para la conversión, pero la simulación exige símbolos de 1 carácter.
La regex muestra ∅	El autómata no tiene estados de aceptación alcanzables: el lenguaje es vacío.
Los botones de edición no responden	Estás en la pestaña Regex con el algoritmo activo; pulsa «Volver al editor».

5. Pruebas realizadas

Pruebas unitarias (JUnit 5, 8 casos): simplificaciones de RegexOps, interpretación de etiquetas, regex conocida del ejemplo de ceros pares (`(1|01*0)*`), normalización, lenguaje vacío, simulación (aceptar/rechazar/ε/símbolo fuera del alfabeto) y detección de no determinismo.

Equivalencia exhaustiva: para los tres autómatas de ejemplo y las dos estrategias de orden, el DFA y la regex obtenida coinciden en las 8191 cadenas sobre Σ = {0,1} (o {a,b}) de longitud ≤ 12, sin contraejemplos.

Pruebas de interfaz: se ejecutó un guion con eventos reales de ratón sobre el lienzo (crear estados, conectar transiciones con diálogos, marcar aceptación, simular, convertir y verificar) y se revisaron las capturas de cada etapa.

Ejemplo (Σ)	Regex obtenida (orden automático)
Cantidad par de 0s ({0,1})	(1|01*0)*
Terminan en «ab» ({a,b})	b*aa*b((a|bb*a)a*b)*
Binarios múltiplos de 3 ({0,1})	(0|1(01*0)*1)*